

파이썬 환경 (for Mac/Windows)

Part1. 운영체제 개념과 터미널 명령어

1. 운영체제란	1
2. 리눅스운영체제	2
3. 윈도우 환경에서 wsl설치하여 리눅스 환경 세팅	4
4. 리눅스명령어	6
5. 조사하기	9
6. 리눅스의 '일정 스케줄러(자동실행 도구)	11

Part2. 파이썬 가상환경

1. WSL에서 python3.10 설치하기	12
2. 주피터랩 실행하기	16
3. VS코드가 설치되어 있다면	17

Part1. 운영체제 개념과 터미널명령어

1 운영체제(OS)란?

- 컴퓨터의 모든 자원을 관리하는 관리자 프로그램
- CPU·메모리·저장장치·네트워크·파일관리·프로그램 실행 등을 대신 처리
- 사용자는 OS 위에서 프로그램을 편하게 사용

○ Windows

- 가장 대중적 / 기업·가정용에서 압도적으로 많이 사용
- 설치·사용이 쉽고 GUI 친화적
- 오피스, 게임, 각종 국내 프로그램 호환성이 최고
- 단점: 무겁고, 업데이트 간섭 많고, 개발(특히 서버·AI) 용도로는 제한 있음

○ Linux

- 서버·개발·AI 환경의 표준 운영체제
- Ubuntu, CentOS, Debian 등 여러 배포판
- 가볍고 안정적, 명령어 기반
- 무료, 커스터마이징 자유로움
- 단점: 초보자에게는 **명령어 중심이라 어렵게 느껴짐**

○ macOS

- 애플(Macbook, iMac)을 위한 운영체제
- **유닉스 기반**이라 개발자에게 매우 친숙 (리눅스와 구조가 비슷)
 - 개발환경(Unix 기반) → **Python, pip, venv, Git 모두 리눅스처럼 작동**
- UI 깔끔, 안정성 좋고 최적화 강점
- iOS 개발 필수 플랫폼
- 단점: 맥북이 비싸고, 일부 윈도우 전용 프로그램 실행 어려움

2 리눅스(Linux) 운영체제

리눅스는 **오픈 소스** 운영 체제(OS)의 일종으로, 서버와 같은 대규모 시스템에서 많이 쓰입니다. 유닉스(Unix) 계열 OS와 비슷하며, 컴퓨터를 제어하는 핵심 소프트웨어입니다.

빅데이터나 AI와 같은 전문 분야는 대부분 **리눅스 기반의 서버 환경**에서 작업이 이루어집니다. 윈도우에서는 **WSL(Windows Subsystem for Linux)**을 통해 리눅스 환경을 쉽게 구축할 수 있습니다.

리눅스는 모든 명령을 타이핑하는 **CLI(Command-Line Interface) 방식**과 윈도우처럼 마우스를 이용해 아이콘을 클릭하고 창을 조작하는 **GUI(Graphical User Interface: 우분투(Ubuntu), 리눅스 민트(Linux Mint)** 등) 환경이 있습니다.

리눅스관련 명령 및 에러는 gpt에서 너무너무너무 잘 알려주며 개발자들도 작업에서 자주 사용하는 명령어는 매우 한정적인 명령어만 외우고 검색하며 사용함
(윈도우랑 생태계가 좀 달라서 처음 접하는 사람은 꾸준히 사용해봐야함, 매일매일 사용해보세요)

→ gpt 검색시 배포판을 명확하게 명시해야함. '우분투2.4의 pip 방법'

- **정의:** 컴퓨터 운영 체제의 핵심인 커널(Kernel)을 의미합니다. 리눅스 커널은 하드웨어와 소프트웨어를 연결하고 관리하는 역할을 합니다.
- **오픈 소스:** 누구나 자유롭게 사용, 수정, 배포할 수 있습니다.
- **다양성:** 리눅스 커널 위에 여러 가지 프로그램과 사용자 환경을 추가하여 다양한 **리눅스 배포판**이 만들어 집니다.

(1) 주요 리눅스 배포판 (대강 쏙 보기)

리눅스 배포판은 크게 3가지 계열로 나눌 수 있으며, 각기 다른 특징을 가집니다.

→ 데비안(Debian) 계열

- **특징:** 안정성을 중요하게 생각하며, 초보자도 쉽게 사용할 수 있는 배포판이 많습니다. `apt` 라는 패키지 관리 도구를 사용합니다.
- **대표 배포판:**

배포판	특징
우분투 (Ubuntu)	데비안 기반으로, 사용자 친화적인 환경과 풍부한 자료 덕분에 개인과 기업 모두에게 인기가 많습니다.
리눅스 민트 (Linux Mint)	우분투를 기반으로 만들어졌으며, 윈도우와 비슷한 사용자 환경을 제공하여 리눅스 입문자에게 적합합니다.

→ 레드햇(Red Hat) 계열

- **특징:** 기업용 서버 환경에 특화되어 있으며, 안정성과 기술 지원을 중요시합니다. `yum` 또는 `dnf` 패키지 관리 도구를 사용합니다.
- **대표 배포판:**

배포판	특징
페도라 (Fedora)	레드햇의 실험적인 버전으로, 최신 기술을 빠르게 도입합니다. 개발 환경에 많이 사용됩니다.
알마리눅스 (AlmaLinux), 록키 리눅스 (Rocky Linux)	과거에 많이 쓰이던 CentOS의 역할을 이어받은 배포판으로, 기업 환경에서 RHEL의 무료 대안으로 사용되고 있습니다.

→ 아치(Arch) 계열

- **특징:** 사용자가 직접 모든 것을 설정하는 'DIY' 방식이며, 항상 최신 버전의 소프트웨어를 제공합니다.
- **대표 배포판:**

배포판	특징
아치 리눅스 (Arch Linux)	설치와 관리가 어렵지만, 시스템을 완벽하게 제어할 수 있어 전문가에게 인기가 많습니다.
만자로 (Manjaro)	아치 리눅스보다 설치가 쉽고 사용이 편리하여 아치 계열을 체험하고 싶은 사용자에게 좋습니다.

(2) 리눅스가 빅데이터 및 AI 작업자에게 중요한 이유

빅데이터와 AI 분야에서 리눅스는 사실상 **표준**으로 자리 잡았습니다. 여러 가지 이유가 있지만, 핵심적인 이유는 다음과 같습니다.

- **서버 환경의 표준:**
빅데이터와 AI 작업은 대부분 **고성능 서버나 클라우드** 환경에서 이루어집니다. 이들 환경은 리눅스 기반으로 구축되는 경우가 압도적으로 많았기 때문에 리눅스에 익숙해야 작업을 효율적으로 진행할 수 있음
- **강력한 커맨드 라인(CLI):** 리눅스는 GUI(그래픽 사용자 인터페이스)보다 **커맨드 라인 인터페이스**를 통한 작업이 훨씬 강력하고 효율적입니다. 수많은 파일과 데이터를 일괄적으로 처리하거나, 복잡한 작업을 스크립트로 자동화할 때 CLI는 필수적입니다. 빅데이터 처리는 대량의 데이터를 다루는 일이므로, CLI의 효율성은 매우 중요합니다. (예: **저녁 몇시에 로그데이터 수집하고 분석해등 일괄처리 명령**)
- **뛰어난 성능과 안정성:** 리눅스는 오랜 기간 검증된 안정성과 높은 성능을 자랑합니다. 특히 시스템 자원을 효율적으로 관리하기 때문에, 대규모 데이터 처리나 복잡한 딥러닝 모델 학습과 같은 **고부하 작업**에 매우 적합합니다.
- **다양한 개발 도구와 라이브러리:** 하둡(Hadoop), 스파크(Spark)와 같은 빅데이터 프레임워크나 **텐서플로(TensorFlow)**, 파이토치(PyTorch)와 같은 AI/머신러닝 라이브러리는 대부분 리눅스를 기본 환경으로 지원합니다. 이러한 도구들은 리눅스 환경에서 가장 최적의 성능을 발휘하도록 설계되었습니다.
- **커뮤니티와 생태계:** 리눅스는 활발한 **오픈 소스 커뮤니티**를 통해 지속적으로 발전하고 있습니다. 문제 발생 시 해결책을 찾기 쉽고, 다양한 도구와 솔루션을 쉽게 구할 수 있다는 장점도 있습니다.

따라서 빅데이터나 AI 관련 작업을 하려면 리눅스 환경에 대한 기본 이해와 커맨드 라인 사용법은 필수적인 역량입니다

3 윈도우 환경에서 WSL설치하여 리눅스 환경 세팅

윈도우 환경에서 리눅스를 사용하는 가장 일반적이고 효율적인 방법은 **WSL (Windows Subsystem for Linux)**을 이용하는 것입니다. 윈도우 10 또는 11에서 공식적으로 지원하는 기능으로, 가상 머신을 사용하지 않고도 리눅스 명령줄 도구와 애플리케이션을 직접 실행할 수 있게 해줍니다.

○ WSL이란

WSL은 윈도우와 리눅스를 통합하는 호환성 레이어입니다. 이 기능을 사용하면 윈도우 앱과 리눅스 앱을 동시에 사용하면서 서로의 파일 시스템에 접근할 수 있어, 개발 환경을 유연하게 구축할 수 있습니다.

예를 들어, 윈도우 탐색기에서 리눅스 파일에 접근하거나, 리눅스 터미널에서 윈도우 프로그램을 실행하는 것도 가능합니다.

(1) WSL 설치 및 사용 방법

WSL이 이미 설치되어 있는지 확인하려면, PowerShell을 실행하고 터미널에 `wsl --list --verbose` 또는 `wsl -l -v` 명령어를 입력하여 설치된 리눅스 배포판 목록을 확인할 수 있습니다.

○ 맥에서 터미널 실행: ⌘ + Space → terminal

`cd ~` / `~` 으로 변경됨 / 맥의 cmd를 **zsh(지셸: Z shell)** 이라함 (user@Mac ~ %)

참고: Unix 계열에서 유명한 셸들이 있음:이 중 **zsh** 는 기능이 제일 많고 편함.

macOS Catalina(2019년 이후)부터 **기본 셸을 bash → zsh 로 변경함.**

- `sh` = Bourne Shell
- `bash` = Bourne Again Shell
- `zsh` = Z Shell
- `fish` = Friendly Interactive Shell

○ 윈도우는 WSL을 설치하여서 리눅스 운영체제사용

- 파워셸(PowerShell) 을 관리자권한으로 실행하고 → 설치된 wsl 확인

PS C:\WINDOWS\system32> wsl --list --verbose

```
C:\Users\bakpa>wsl --list --verbose
NAME                STATE              VERSION
* Ubuntu            Stopped            2
```

○ WSL이 없다면 설치

1. **설치 명령어 실행:** 관리자 권한으로 파워셸(PowerShell) 또는 명령 프롬프트(Command Prompt)를 열고, `wsl --install` 명령어를 입력합니다. 이 명령은 필요한 모든 구성 요소를 자동으로 설치하고, 기본 리눅스 배포판(보통 Ubuntu)을 설치합니다.
2. **재부팅:** 설치가 완료되면 컴퓨터를 재부팅합니다.
3. **리눅스 환경 설정:** 재부팅 후, 자동으로 리눅스 터미널 창이 열립니다.
또는 wsl 실행하여 사용자 이름과 비밀번호(**타이핑할때 화면에 글자표시되지 않음**)를 설정하면 리눅스 환경에 접속할 수 있습니다. 이제 `ls`, `cd`, `sudo` 와 같은 리눅스 명령어를 사용할 수 있습니다. (또는 윈도우에서 wsl 검색하여 실행)

WSL 사용의 장점

- **높은 성능:** 기존의 가상 머신(VirtualBox, VMware 등)에 비해 훨씬 가볍고 빠름
- **원활한 통합:** 윈도우와 리눅스 파일 시스템을 함께 사용할 수 있어 개발 작업의 효율성이 높아짐
- **손쉬운 설치:** 단일 명령어로 손쉽게 설치 및 설정할 수 있음

WSL 외에 가상 머신 프로그램을 사용하는 방법도 있지만, 대부분의 개발 작업에서는 WSL이 더 효율적이고 편리한 솔루션으로 여겨지고 있습니다.

4 리눅스 명령어

- pwd: print working directory

사용자 이름이 **bu** 라고 가정할 때,

맥(Mac) 터미널과 **WSL(Ubuntu on Windows) 터미널**

사용자 이름이 **bu**라면:

- 맥: /Users/bu
- WSL: /home/bu

둘 다 `cd ~` 또는 `cd` 하면 그 위치로 이동.

Mac 터미널 (zsh 기준)

bu@MacBook-Air ~ %

`cd ~` 또는 `cd` 하면:

bash

bu@MacBook-Air ~ %

`pwd`

/Users/bu

현재경로확인

WSL(Ubuntu) 터미널 (bash 기준)

bu@DESKTOP-AAAAAAA:~\$

`cd ~` 또는 `cd` 하면:

ruby

bu@DESKTOP-AAAAAAA:~\$

`pwd`

/home/bu

• 폴더/파일 목록 보기

명령어	설명
<code>ls</code>	파일 목록 보기
<code>ls -l</code>	상세 정보 보기 (권한/크기/날짜)
<code>ls -a</code>	숨김파일 포함
<code>ls -lh</code>	사람이 읽기 쉬운 크기로 표시

• 폴더(디렉토리) 만들기

- `mkdir test`

userbu@DESKTOP-96N9HJ3:~\$ `cd test`

userbu@DESKTOP-96N9HJ3:~/test\$

명령어	설명
<code>mkdir</code> 폴더명	새 폴더 생성
<code>mkdir -p a/b/c</code>	경로 전체를 한 번에 생성

- 리눅스(WSL/우분투)에서 파일 또는 폴더의 권한(permission)위치 이동 관련 명령어

폴더이며, 소유자는 읽기·쓰기·실행 가능, 다른 사람은 읽기·실행만 가능.

```
/content# ls -l  
total 4  
drwxr-xr-x 1 root root 4096 Nov 20 14:30 sample_data
```

문자	의미
d	directory (폴더)
r	read (읽기)
w	write (쓰기)
x	execute (실행)

1) d

→ 디렉토리(폴더)

2) rwx (Owner, 소유자 권한)

→ 읽기 + 쓰기 + 실행 가능

3) r-x (Group, 그룹 권한)

→ 읽기, 실행 가능

→ 쓰기는 불가(-)

4) r-x (Other, 그 외 사용자)

→ 읽기, 실행 가능

→ 쓰기는 불가(-)

• 폴더이동 등

명령어	설명
pwd	현재 위치 확인
cd 폴더명	해당 폴더로 이동
cd ..	상위 폴더로 이동
cd ~	홈 디렉토리로 이동
cd /경로/경로	절대경로 이동
cd -	이전 디렉토리로 이동

• 파일 만들기 / 보기

명령어	설명
touch 파일명	비어있는 새 파일 생성
cat 파일명	파일 내용 출력
nano 파일명	터미널 편집기로 파일 열기

```

userbu@DESKTOP-96N9HJ3:~/test$ touch a.txt
userbu@DESKTOP-96N9HJ3:~/test$ ls
a.txt
userbu@DESKTOP-96N9HJ3:~/test$ cat a.txt
userbu@DESKTOP-96N9HJ3:~/test$ nano a.txt
userbu@DESKTOP-96N9HJ3:~/test$ cat a.txt
.... python
userbu@DESKTOP-96N9HJ3:~/test$
        
```

(1) 내용입력

(2) Ctrl+O 후 Enter

(3) Ctrl + X (종료)

○ 구글코랩에서는 nano → mcedit (Midnight Commander 편집기) 사용

- 정리하세요 → [코랩에서 터미널을 사용하면 좋은이유] [리눅스에서 apt 란]

```
sudo apt install mc -y
```

```
mcedit a.txt
```

내용입력 예: ...python → (저장) ctrl+S → (종료) ctrl+Q

○ 정리하세요: mac에서의 sudo

macOS에서는 apt 를 아예 사용할 수 없다.

macOS는 Ubuntu/Debian 기반이 아니라서 apt , apt-get , dpkg 같은 패키지 관리 도구가 존재하지 않음.

✓ Homebrew(brew) 를 사용해야 한다.

macOS = BSD Unix 기반

Ubuntu = Debian Linux 기반

→ 패키지 시스템 자체가 완전히 다름.

OS	패키지 관리자
Ubuntu / Debian	apt, dpkg
CentOS / RHEL	yum, dnf
Arch Linux	pacman
macOS	brew
Windows	winget, choco

• 파일/폴더 복사

명령어	설명
cp 파일1 파일2	파일 복사
cp -r 폴더1 폴더2	폴더 전체 복사

• 파일폴더 이름변경

명령어	설명
mv 기존 새이름	파일/폴더 이름 변경
mv 파일 경로	다른 폴더로 이동

• 파일폴더삭제 (주의)

명령어	설명
rm 파일명	파일 삭제
rm -r 폴더명	폴더 삭제
rm -rf 폴더명	강제 삭제 (절대 조심!!)

```
userbu@DESKTOP-96N9HJ3:~/test$ ls
a.txt
userbu@DESKTOP-96N9HJ3:~/test$ cp a.txt b.txt
userbu@DESKTOP-96N9HJ3:~/test$ ls
a.txt b.txt
userbu@DESKTOP-96N9HJ3:~/test$ cat b.txt
.... python
userbu@DESKTOP-96N9HJ3:~/test$ mv b.txt bb.txt
userbu@DESKTOP-96N9HJ3:~/test$ ls
a.txt bb.txt
userbu@DESKTOP-96N9HJ3:~/test$ find *.txt
a.txt
bb.txt
userbu@DESKTOP-96N9HJ3:~/test$ grep "py" a.txt
.... python
userbu@DESKTOP-96N9HJ3:~/test$ |
```

• 경로찾기/검색

명령어	설명
find . -name "*.py"	현재 폴더에서 .py 파일 찾기
which python	파이썬 실행 경로 찾기
grep "문자" 파일	파일 안에서 문자열 찾기

• 시스템 관련 기본 명령어

명령어	설명
top	실시간 프로세스 보기
htop	top 개선 버전(설치 필요)
df -h	디스크 사용량 보기
du -h	현재 폴더 용량 보기
ps aux	모든 프로세스 표시
whoami	현재 로그인 사용자

• sudo 관련

명령어	설명
sudo 명령어	관리자 권한으로 실행
sudo apt update	패키지 목록 업데이트
sudo apt install 이름	프로그램 설치

(참고: 다음 표안의 명령어 복사가능함)

여러 개 명령을 한 줄로 자동 실행	<code>clear && ls && pwd</code>
폴더만들기	<code>mkdir -p dataset/{images,labels}/{train,val,test}</code>

:: -p 옵션: 부모 폴더까지 전부 자동으로 생성

```
userbu@DESKTOP-96N9HJ3:~$ mkdir -p dataset/{images,labels}/{train,val,test}
userbu@DESKTOP-96N9HJ3:~$ ls
'1. test_yolo.ipynb'  background.jpg  py310  test.py  untitled.py
Untitled.ipynb       dataset        test    test_st.py  yolov8n.pt
userbu@DESKTOP-96N9HJ3:~$ cd dataset
userbu@DESKTOP-96N9HJ3:~/dataset$ ls
images  labels
userbu@DESKTOP-96N9HJ3:~/dataset$ cd images
userbu@DESKTOP-96N9HJ3:~/dataset/images$ ls
test  train  val
userbu@DESKTOP-96N9HJ3:~/dataset/images$ cd..
cd..: command not found
userbu@DESKTOP-96N9HJ3:~/dataset/images$ cd ..
userbu@DESKTOP-96N9HJ3:~/dataset$ cd labels
userbu@DESKTOP-96N9HJ3:~/dataset/labels$ ls
test  train  val
userbu@DESKTOP-96N9HJ3:~/dataset/labels$ |
```

- 연습한 폴더 삭제: `cd ~` 로 home 이동후 `rm -rf dataset`

구글코랩에서는 `cd ~` 하면 root로 이동함. `cd /content/` 등으로 다시 이동하고자 하는 폴더를 지정함

5 조사하기

- 상대경로 / 절대경로
- grep를 이용한 파일 찾기
- curl / wget (URL로 파일 받기)
- ps / kill (프로세스 강제 종료)
- history (전 명령 보기)
- tar / unzip (압축 파일 처리)

6 리눅스의 "일정 스케줄러(자동 실행 도구) 예: WSL에서 사용

`crontab` 은 리눅스에서 특정 작업을 정해진 시간에 자동으로 실행시키는 윈도우의 작업 스케줄러 (Task Scheduler)와 같은 개념.

✓ `crontab` = cron table

- `cron` = 백그라운드에서 주기적으로 작업을 실행하는 데몬(process)
- `crontab` = 실행할 작업 목록(테이블)

시간 맞춰 자동 실행 = cron

cron은 1분 단위 : 1초 단위 원하면 watch 사용.

* * * * * 실행할명령

`crontab -e`

| | | | |
| | | | └─ 요일 (0~6, 일~토)
| | | └─── 월 (1~12)
| | └───── 일 (1~31)
| └──────── 시 (0~23)
└───────── 분 (0~59)

- 매일 3시에 자동 백업하기
- 매일 새벽 1시에 YOLO 학습 재시작
- 5분마다 서버 상태 체크
- 매 10초마다 로그 자동 저장
- 매시간 캐시 비우기

→ 실습: 샘플작업

- `a.txt` 를 매 분마다 자동으로 백업
- 백업 파일명에 분 정보만 붙임
- 폴더 안 만들기
- 파일이 계속 쌓임 → `a_00.txt` , `a_01.txt`

○ 1분뒤 확인해보면

```
userbu@DESKTOP-96N9HJ3:~/test$ ls  
a.txt a_1803.txt a_1804.txt a_1805.txt bb.txt
```

- 자동화작업 작성법

crontab -e 로 실행한 자료에 붙여넣을 코드

```
* * * * * cp /home/본인계정/test/a.txt /home/본인계정/test/a_$(date +%H%M).txt
```

The image shows a terminal window with the following content:

```
userbu@DESKTOP-96N9HJ3:~/dataset/labels$ crontab -e
no crontab for userbu - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano          <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab
```

Below this, the nano editor is shown editing `/tmp/crontab.jME` with the following content:

```
GNU nano 7.2 /tmp/crontab.jME
* * * * * cp /home/bu/a.txt /home/bu/a_$(date +%H%M).txt
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
```

Numbered callouts point to specific actions:

- 1: `crontab -e`
- 2: Choosing editor `1` (nano)
- 3: Pasting the cron job line into the editor

A button at the bottom right indicates the exit sequence: `Ctrl+O -> Enter -> Ctrl+X`.

- crontab서비스시작: `sudo service cron start`

일반 Linux(우분투)에서는 `cron` 데몬은 부팅 시 자동으로 실행됨.

- `crontab -e` 로 스케줄 추가 / 저장 후 자동 실행됨 / 별도의 start 불필요

WSL에서만 예외: 반드시 처음 한 번 `sudo service cron start` 해야함

- crontab 목록리스트확인: `crontab -l`
- 모든 crontab 삭제: `crontab -r`
`userbu@DESKTOP-96N9HJ3:~/test$ crontab -l`
`no crontab for userbu`
- `wsl --shutdown` 하면 wsl에서는 crontab 도 종료됨

Part2. 파이썬 가상환경

○ 가상환경 정의

프로젝트마다 파이썬을 독립된 공간으로 따로 만들어서, 서로 충돌하지 않게 해주는 격리된 파이썬 환경" → 프로젝트마다 "독립된 파이썬 세상"이 필요

기능	설명
라이브러리 충돌 방지	프로젝트별 numpy, torch 버전 분리
깔끔한 환경 유지	pip 리스트가 프로젝트마다 독립
재현성 확보	requirements.txt로 동일 환경 재구축
안전함	시스템 파이썬 망칠 걱정 없음

1 WSL 에서 python3.10 설치하기

우분투는 시스템 기본 python3(3.12 등)를 건드리면 OS 오류가 날수 있기때문에 ,Python 3.10을 따로 설치 후, 원하는 프로젝트에서만 사용하는 방식이 가장 좋습니다.

- 설치된 파이썬 버전 확인: `which python3` 또는 `python3.10 --version`
 - 설치된 우분투버전확인하기: `lsb_release -a`
- 파이썬 3.10 설치
 - 우분투 기본 저장소에는 없는 여러 버전의 Python을 설치할 수 있도록, 외부 소스(PPA)를 시스템에 등록하는 작업 → 업데이트 → 파이썬3.10 설치 → 파이썬 버전확인
 - 참고: sudo: 리눅스.맥에서 관리자(root) 권한으로 명령을 실행하는 명령어

○ windows

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install python3.10 python3.10-venv python3.10-dev -y
python3.10 --version
```

○ Mac

```
brew update
brew install python@3.10
brew link python@3.10
python3.10 --version
```

○ 참고: 우분투에 "새로운 소프트웨어 저장소(Repository)"를 추가하는 명령

특히 Python 최신 버전 설치할 때 자주 쓰는 PPA가 바로 `deadsnakes`.

```
sudo add-apt-repository ppa:deadsnakes/ppa -y
```

요소	뜻
sudo	관리자(root) 권한으로 실행
add-apt-repository	apt가 사용할 새로운 저장소를 추가
ppa:deadsnakes/ppa	deadsnakes 라는 PPA 저장소 추가
-y	모든 질문에 자동으로 "Yes"

PPA란: Personal Package Archive (개인 패키지 아카이브)

- 우분투 공식 저장소에는 없는 패키지
- 또는 더 최신 버전의 패키지
- 개발자가 직접 유지하는 저장소

예: Python 최신버전 제공 / GPU 라이브러리 최신 버전 / 특수 패키지

○ deadsnakes PPA

Ubuntu에서 공식 제공하지 않는 Python 최신/구버전을 제공해주는 유명한 저장소

*표안 텍스트 복사가능 (windows)

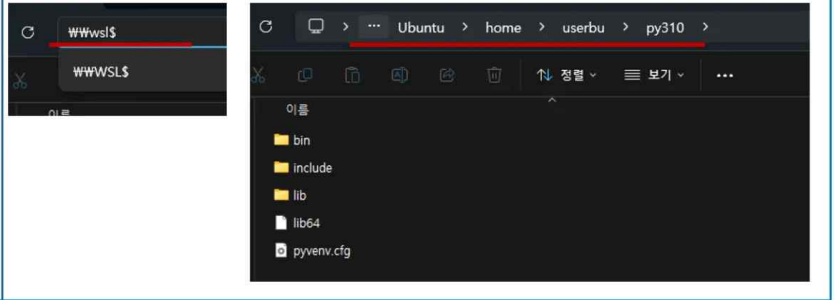
```
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install python3.10 python3.10-venv python3.10-dev -y

python3.10 --version
```

*표안 텍스트 복사가능 (mac)

```
brew update
brew install python@3.10
brew link python@3.10
python3.10 --version
```

- **which python** : 사용하는 파이썬 경로 확인
- **which pip**
- **파이썬 3.10 전용 가상환경 만들기**
 - `python3.10 -m venv py310`
 - `-m`: 모듈(Module)을 실행 / `venv`: 파이썬 내장 가상환경 생성 모듈.

가상환경 만들기 	윈도우 탐색기에서 <code>\\wsl\$</code> 하면 ubuntu 시스템으로 접속가능, 생성한 가상환경 py310을 확인할수 있음. 
---	---

- `source py310/bin/activate`: py310 가상환경을 활성화 → (화면좌측에py310가상환경이름 표시됨)
- **which python** 로 사용하는 파일 확인 (home/user계정/py301가상환경명/bin/pip)
- 모듈설치

*표안 텍스트 복사가능

```
pip install ultralytics  
pip install torch torchvision torchaudio  
pip install matplotlib pandas streamlit  
pip install opencv-python  
pip install jupyterlab
```

📌 Jupyter Notebook (주피터 노트북)

- 셀 단위로 코드를 실행하는 **간단한 실습용 도구** / 파일 하나만 열어서 작업함 / UI가 단순함
- 교육·데이터 분석 입문용에 적합

📌 JupyterLab (주피터 랩)

- Notebook의 **업그레이드 버전** / VSCode처럼 **탭, 파일탐색기, 터미널, 여러 창 레이아웃** 지원
- 프로젝트 작업, 딥러닝 개발에 적합 / 확장 기능 풍부(Git, 변수창 등)

구분	Jupyter Notebook	JupyterLab
UI 구조	단일 화면, 탭 없음	VSCode 같은 멀티 탭·레이아웃
파일 탐색기	✗ 없음	✓ 좌측 파일 탐색기 기본 제공
레이아웃 분할	불가능	✓ 창 가로/세로 자유 배치
터미널 통합	✗ 없음	✓ 터미널 열기 가능
콘솔(Console)	없음	✓ 별도 코드 콘솔 실행 가능
다중 파일 작업	불편 (노트북 하나씩만 열림)	✓ 여러 노트북·py·md·csv 파일 동시 작업
확장성	매우 제한적	✓ Git UI, Variable Inspector 등 확장 풍부
UI 모드	라이트만 기본	✓ Dark Mode/Light 기본 지원
데이터 파일 뷰어	CSV/JSON 제한적으로 표시	✓ CSV/JSON/HDF5 이미지 등 강력한 뷰어
프로젝트 작업	단일 파일 중심	✓ 프로젝트 구조에 적합
개발 편의성	기본 수준	✓ 코드 에디터 기능 향상
권장도	초급 실습용	✓ 현업/ML 개발 기본 환경
공식 방향성	유지(LTS 수준), 확장성 낮음	✓ Jupyter 공식 최신 표준(후속 버전)

- 가상환경내의 모듈(버전포함) 정보다운받기

```
pip freeze > requirements.txt
```

- 다른 환경에서 복원 할때는 'requirements.txt' 파일이 있는 위치에

```
pip install -r requirements.txt
```


2 주피터랩 실행하기

- WSL가상환경에서 (py30) jupyter lab

```
import streamlit as st
from ultralytics import YOLO
from PIL import Image
import numpy as np
import cv2

# 모델 로드 (YOLOv8n: 가장 가벼운 모델)
model = YOLO("yolov8n.pt")

st.title("YOLO Object Detection Demo (초간단 버전)")

uploaded_file = st.file_uploader("이미지를 업로드하세요.", type=["jpg", "png", "jpeg"])

if uploaded_file:
    img = Image.open(uploaded_file)
    st.image(img, caption="원본 이미지", use_container_width=True)

    # YOLO 예측
    results = model.predict(img)

    # 결과 이미지 (박스 그려진 이미지)
    result_img = results[0].plot() # numpy array
    result_img = cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB) # RGB 변환

    st.image(result_img, caption="YOLO 탐지 결과", use_container_width=True)
```

/home/userbu/py310/ ← 라이브러리, pip 패키지만 들어 있는 공간
/home/userbu/yolo_streamlit/ ← test.py, app.py 등 py코드

3 VScode가 설치되어 있다면

Visual Studio Code (VS Code)는 Microsoft에서 개발한 무료로 사용할 수 있는 오픈 소스의 소스 코드 편집기로 그 유연성과 강력한 기능 덕분에 현재 전 세계 개발자들에게 가장 인기 있는 코드 편집기 중 하나입니다.

주요 특징 ✨

- **가볍고 빠름:** 기존의 무거운 IDE(통합 개발 환경)와 달리 매우 빠르게 실행되고 작동합니다.
- **다양한 언어 지원:** JavaScript, TypeScript, Python, C++, Java 등 수많은 프로그래밍 언어를 기본적으로 지원하며, 확장을 통해 더 많은 언어를 사용할 수 있습니다.
- **강력한 기능:**
 - IntelliSense (지능형 코드 완성 기능)
 - 디버깅 (코드 오류를 찾고 수정하는 기능)
 - Git 통합 (버전 관리 시스템을 쉽게 사용할 수 있음)
- **확장성:** 수많은 확장 프로그램(Extensions)을 통해 기능을 추가하고 개인화할 수 있어, 개발 환경을 원하는 대로 커스터마이징할 수 있습니다.
- **크로스 플랫폼:** Windows, macOS, Linux 등 모든 주요 운영 체제에서 실행됩니다.

VS Code 설치 시 WSL 확장(VS Code Remote - WSL)이 자동으로 설치되어 있어 WSL 내부 폴더를 Windows VS Code에서 바로 열 수 있음.

VS Code가 Windows에 이미 설치되어 있다면 WSL(Ubuntu) 터미널에서 그냥:

`code .`

하면 바로 VS Code가 열리고, 지금 있는 WSL 폴더를 그대로 VS Code로 불러온다.

만약 `code: command not found` 뜨면?

그러면 VS Code 설치 시 PATH 등록이 빠진 것. 이럴 때는 VS Code에서:

- **Ctrl + Shift + P**
- "Shell Command: Install 'code' command in PATH" 선택

설치하면 WSL에서도 바로 `code` 명령 사용 가능.